RENTGUARD RWANDA: RENTAL BILLING AND RENT ARREARS TRACKER

Student Name: Ndayisenga Kevin
Student ID: 29397
Course: INSY 8311 – Database Development with PL/SQL
Institution: Adventist University of Central Africa (AUCA)
Academic Year: 2026–2027

1. Problem Statement

Managing rental payments can be difficult for landlords, especially when they have many tenants and properties. Some landlords use notebooks, paper records, or phone messages to keep track of rent payments.

This method can cause problems. A landlord may forget to record a payment, calculate the wrong balance, or fail to notice that a tenant has not paid rent on time. Finding old payment records can also take a lot of time.

This project aims to solve these problems by developing a database system that makes rental payment management easier.

2. Proposed Solution

RentGuard Rwanda will be developed using Oracle Database and PL/SQL. It will help landlords keep rental information in one organized place.

The system will store information about landlords, properties, rental units, tenants, leases, invoices, and payments.

It will also calculate the amount of rent paid, the remaining balance, and any applicable late fees. Landlords will be able to identify tenants who have not completed their rent payments.

3. Data Source

The project will use the House Rent Prediction Dataset available on Kaggle.

Link: https://www.kaggle.com/datasets/iamsouravbanerjee/house-rent-prediction-dataset

The dataset contains information about rental properties, including location, house size, number of bedrooms, furnishing status, and rental prices.

However, the dataset is based on Indian rental listings and does not contain actual Kigali tenant or payment records. For database testing, I may create sample tenant, lease, invoice, and payment records. These records will be identified as fictional test data.

4. Planned Database Tables

The database will have the following tables:

- LANDLORD – stores landlord information.
- PROPERTY – stores property information.
- UNIT – stores individual rental units.
- TENANT – stores tenant information.
- LEASE – stores rental agreements.
- RENT_INVOICE – stores rent invoices.
- PAYMENT – stores payment records.
- LATE_FEE_RULE – stores rules for late payment charges.
- ARREARS_ALERT – stores information about overdue rent.
- AUDIT_LOG – keeps records of important changes.

The tables will be connected using primary keys and foreign keys to organize the information and maintain data accuracy.

5. Planned PL/SQL Features

I will use PL/SQL to perform the main activities of the system.

- **Procedures:** To create rental agreements and record payments.
- **Functions:** To calculate late fees and the remaining rent balance.
- **Cursors:** To process active rental agreements.
- **Triggers:** To prevent overlapping leases and invalid payments.
- **Packages:** To organize related procedures and functions.
- **Exception handling:** To manage errors such as invalid payment amounts.
- **Audit logs:** To record important database changes.
- **DBMS_SCHEDULER:** To check overdue invoices automatically.

6. Innovation

One useful feature of RentGuard Rwanda will be automatic checking of overdue rent.

I plan to use Oracle DBMS_SCHEDULER to run a scheduled database task. This task will check invoices that have not been paid by their due dates, update their status, calculate applicable late fees, and create alerts.

This will reduce the need for landlords to check every payment manually.

7. References

1. Bassett and Pisano (2022). *Property Management – An Industry of Fragmented Technology Solutions.* https://doi.org/10.7250/itms-2022-0002
2. National Institute of Statistics of Rwanda (2025). *EICV7 Main Indicators Report.* https://www.statistics.gov.rw/sites/default/files/documents/2025-07/EICV7_Main%20Indicator%20Report.pdf
3. Banerjee, S. *House Rent Prediction Dataset.* Kaggle. https://www.kaggle.com/datasets/iamsouravbanerjee/house-rent-prediction-dataset
4. Oracle. *PL/SQL Subprograms and Packages.* https://docs.oracle.com/en/database/oracle/oracle-database/26/adfns/coding-subprograms-and-packages.html